FieldGuard Pro — Architecture Guide

Deep-Dive Engineering Guide: Angular 22, Ionic 9 & Capacitor 8

Author: Antigravity Engineering • **Audience:** Engineers, Architects, Technical Leads
Domain: Industrial Field Inspection, Asset Maintenance & Offline Reliability

1. Executive Overview & System Purpose

FieldGuard Pro is a mobile-first enterprise progressive web and hybrid mobile application designed for harsh, network-constrained environments. Target industries include **Manufacturing, Oil & Gas, Electric Utilities, Construction, Telecom**, and **Healthcare Facilities**. In these environments, field engineers must inspect high-value equipment (centrifugal pumps, gas turbines, step-down transformers, compressors, piping) where cellular or Wi-Fi connectivity is spotty or nonexistent.

To solve this problem, FieldGuard Pro is constructed around an **Offline-First Architecture**: the application treats local on-device persistence as its primary, immediate source of truth. When the device is offline, inspections, photo captures, and digital sign-offs are queued locally. When connectivity returns, a background synchronization engine pushes changes to the remote server and reconciles audit state.

CORE VALUE PROPOSITION: Zero data loss in the field, zero blocking spinners when network degrades, and full traceability with role-based supervisor review workflows.

---

2. Application Lifecycle: Where Does the Code Begin?

Understanding how a modern **Angular 22 Standalone** application boots within the **Ionic** and **Capacitor** ecosystem is essential. Below is the precise lifecycle sequence from browser index load to active component view:

2.1 The HTML Entrypoint (src/index.html)

Execution begins in index.html. Notice the mobile-specific viewport configuration:

```
<meta name="viewport" content="viewport-fit=cover, width=device-width, initial-scale=1.0,
minimum-scale=1.0, maximum-scale=1.0, user-scalable=no" /> <link rel="icon" type="image/png"
href="assets/icon/favicon.png" /> <app-root></app-root>
```

- **viewport-fit=cover**: Informs iOS Safari and Android WebView to render underneath notches, dynamic islands, and home indicator bars. Ionic CSS variables automatically inset toolbar contents safely.
- **user-scalable=no**: Prevents double-tap zooming on mobile touch interactions, maintaining a native-feeling touch response.

2.2 Bootstrap Entrypoint (src/main.ts)

In modern Angular, there is no legacy AppModule. Instead, the application initializes via bootstrapApplication() in main.ts:

```
import { bootstrapApplication } from '@angular/platform-browser'; import { RouteReuseStrategy, provideRouter, withComponentInputBinding, withPreloading, PreloadAllModules } from '@angular/router'; import { IonicRouteStrategy, provideIonicAngular } from '@ionic/angular'; import { defineCustomElements } from '@ionic/pwa-elements/loader'; // 1. Initialize PWA Elements for Camera support in web/desktop defineCustomElements(window); // 2. Bootstrap root AppComponent with providers bootstrapApplication(AppComponent, { providers: [ { provide: RouteReuseStrategy, useClass: IonicRouteStrategy }, provideIonicAngular(), provideRouter(routes, withPreloading(PreloadAllModules), withComponentInputBinding()), ], });
```

Critical Bootstrap Responsibilities:

1. **defineCustomElements(window)**: Loads @ionic/pwa-elements. When Capacitor Camera is executed on web browsers (development or desktop PWA), it mounts a custom camera modal element into the DOM instead of crashing.
2. **provideIonicAngular()**: Configures Ionic runtime animations, hardware back-button listener, platform detection (iOS/Android/desktop), and overlay services.
3. **RouteReuseStrategy -> IonicRouteStrategy**: Essential for mobile apps! Unlike standard Angular routing which destroys components on navigation, Ionic caches previously visited views in the DOM so returning to the list preserves scroll position and state.
4. **withPreloading(PreloadAllModules)**: Downloads lazy feature bundles in the background after initial paint, ensuring instant screen transitions in the field.

2.3 The Root Shell Component (src/app/app.component.ts)

The root component contains <ion-app><ion-router-outlet></ion-router-outlet></ion-app>. The <ion-app> element initializes the global CSS gesture system and theming variables. The <ion-router-outlet> coordinates page transition animations (slide horizontal on iOS, subtle zoom/fade on Android).

---

3. Angular 22 Modern Concepts in the Codebase

This project implements the newest idiomatic Angular patterns:

3.1 Standalone Architecture

Every page and component in the codebase is marked standalone: true. No NgModule exists. Each component explicitly declares its dependencies in its imports: [...] array. This provides tree-shaking efficiency, faster compilation times, and clear dependency boundaries.

3.2 Angular Signals (signal, computed, effect)

Instead of complex RxJS subject state management for synchronous UI reactivity, the app uses **Angular Signals**:

```
// src/app/features/dashboard/dashboard.page.ts private inspectionsSignal = signal<Inspection[]>([]); // Computed signals automatically re-calculate only when inspectionsSignal changes public readonly totalCount = computed(() => this.inspectionsSignal().length); public readonly approvedCount = computed(() => this.inspectionsSignal().filter(i => i.status === 'APPROVED').length ); public readonly pendingApprovalCount = computed(() => this.inspectionsSignal().filter(i => i.status === 'PENDING_APPROVAL').length );
```

3.3 Reactive Forms with Dynamic Conditional Validation

In InspectionFormPage, the form enforces real-time conditional validation. When equipment damage is detected, damage description and severity rating dynamically become mandatory:

```
onDamageToggle(): void { const hasDamage = this.inspectionForm.get('hasDamage')?.value; const descControl
= this.inspectionForm.get('damageDescription'); const sevControl = this.inspectionForm.get('severity');
if (hasDamage) { descControl?.setValidators([Validators.required]);
sevControl?.setValidators([Validators.required]); } else { descControl?.clearValidators();
sevControl?.clearValidators(); } descControl?.updateValueAndValidity();
sevControl?.updateValueAndValidity(); }
```

4. Ionic Framework 9 Core Concepts

Ionic Framework transforms Angular into a high-performance cross-platform mobile UI layer:

4.1 Navigation & View Lifecycle

Standard Angular components only have ngOnInit and ngOnDestroy. But mobile navigation maintains view stacks. The codebase leverages **ionViewWillEnter** on pages like DashboardPage, InspectionListPage, and ApprovalListPage. Whenever the user pops back to a page, ionViewWillEnter triggers fresh data fetching from the repository.

4.2 Mobile UI Patterns

- **ion-refresher**: Allows pull-to-refresh on mobile touchscreens with automatic completion.
- **ion-infinite-scroll**: Automatically paginates list results (8 items per chunk) as the user scrolls down.
- **ion-fab & ion-fab-button**: Floating Action Button fixed to the bottom-right corner for quick inspection creation.
- **ion-segment**: Tabbed filter control for instantly filtering between All, Pending, Drafts, Approved, and Rejected.

4.3 Programmatic Overlay Controllers

Rather than rendering popups in the template DOM, Ionic uses imperative controllers:
• **ModalController**: Mounts full-screen modals for ImagePreviewModalComponent and QrScannerModalComponent.
• **AlertController**: Prompts supervisors for approval confirmation and mandatory rejection comments.
• **ActionSheetController**: Presents native bottom-sheet menus for the Sync Manager.
• **ToastController**: Displays top and bottom status notifications with dismiss buttons.

5. Capacitor JS & Native Hardware Plugins

Capacitor is the native container that bridges web JavaScript with iOS and Android operating system APIs:

5.1 Capacitor Camera (@capacitor/camera)

Implemented in CameraService (src/app/core/services/camera.service.ts):

```
const photo = await Camera.getPhoto({ quality: 85, allowEditing: false, resultType:
CameraResultType.DataUrl, source: CameraSource.Camera, // Or CameraSource.Photos for gallery width: 1280,
});
```

• **Permission Handling**: Proactively requests camera and photos permissions via Camera.checkPermissions() and Camera.requestPermissions().
• **Storage Format**: Images are captured as Base64 Data URLs, allowing direct storage in local JSON offline entities without requiring complex local file URI path resolution.
• **Web Fallback**: If native camera is canceled or unavailable, a fallback file input trigger provides instant file upload capability.

5.2 Capacitor Network (@capacitor/network)

Implemented in NetworkService (src/app/core/services/network.service.ts). It uses Network.getStatus() and Network.addListener('networkStatusChange') to track physical link connectivity. Crucially, it introduces a **Simulated Offline Mode**: engineers and testers can toggle offline simulation directly from the header Sync Badge to test offline queueing and recovery on any desktop browser without turning off their Wi-Fi.

5.3 Capacitor Configuration (capacitor.config.ts)

Defines the native application bundle ID (io.ionic.fieldinspector), app title, and web build directory (www).

---

6. Architectural Design Patterns

The codebase demonstrates two industry-standard design patterns required by enterprise specifications:

6.1 The Adapter Pattern (InspectionAdapter)

The **Adapter Pattern** (src/app/core/repositories/inspection.adapter.ts) completely decouples the internal domain model from external backend API DTOs and local persistence formats:

Domain Model (Inspection)	Remote API DTO (InspectionApiDto)	Local Entity (InspectionLocalEntity)
`id`	`uuid`	`id`
`assetId`	`asset_code`	`data (JSON string)`
`hasDamage`	`damage_detected`	`syncStatus`
`notes`	`field_notes`	`updatedAt (timestamp)`
`photos[]`	`attached_images[]`	`-`

6.2 The Repository Pattern

The **Repository Pattern** abstracts data access operations behind the InspectionRepository contract:

1. **LocalInspectionRepository**: Persists entities to on-device storage. Guarantees immediate, synchronous read/write performance in offline conditions.
2. **RemoteInspectionRepository**: Simulates remote backend REST endpoints with realistic network latency (250-300ms) and throws realistic network error exceptions when offline.
3. **SyncInspectionRepository**: The unified orchestrator consumed by UI pages. When reading, it returns local data immediately. When writing online, it synchronizes with the remote API; when writing offline, it writes to local storage and enqueues the action into SyncQueueService.

---

7. Offline Synchronization Queue Engine

The heart of FieldGuard Pro is SyncQueueService (src/app/core/sync/sync-queue.service.ts):

```
+-------------------------------------------------------------------------+ | USER SUBMITS INSPECTION |
+-------------------------------------------------------------------------+ | Is Device Online? / \ (YES)
(NO) / \ Direct POST to API Save locally as PENDING_SYNC Mark local: SYNCED Enqueue item in
SyncQueueService | Device reconnects to network | NetworkService.onStatusChange fires |
SyncQueueService.processQueue() runs | Push each item -> Mark SYNCED locally
```

- **Persistent Queue Storage**: The queue is preserved in localStorage under fia_sync_queue across app reboots.
- **Retry & Fault Tolerance**: If an item fails during upload (e.g. server timeout), its retryCount increments. If failures

exceed 3 attempts, it transitions to SYNC_FAILED and records lastError without blocking subsequent queue items.
• **Manual & Automatic Triggers**: The queue automatically triggers upon network reconnect, or on-demand via the 'Sync Queue Now' button in the SyncBadgeComponent.

8. Role-Based Authentication & Route Guards

The application provides role-based access control (RBAC) across two personas:

Role	Demo User	Email	Capabilities
EMPLOYEE	Alex Carter	alex@fieldservice.com	Create/edit inspections, capture photos, save drafts, submit for review
SUPERVISOR	Sarah Jenkins	sarah@fieldservice.com	Access /approvals portal, review damage, approve/reject with remarks, audit log

• **authGuard**: Protects private routes. Redirects unauthenticated visitors to /login.
• **roleGuard**: Protects the /approvals route. If an Employee attempts to open supervisor approvals, they are blocked and redirected to /dashboard with a toast warning.
• **Session Expiry Handling**: Sessions expire after 8 hours. The service automatically invalidates expired sessions and performs state cleanup on logout.

9. Complete Codebase Directory & File Walkthrough

Here is a complete map of all source files in the project:

```
src/ ■■■ main.ts # Bootstrap entrypoint with PWA elements & Ionic providers ■■■ index.html # HTML shell with mobile viewport & safe-area meta ■■■ theme/variables.scss # Industrial enterprise color palette CSS tokens ■■■ app/ ■ ■■■ app.routes.ts # Route definitions with lazy loading & guards ■ ■■■ app.component.ts # Root component (<ion-app> & <ion-router-outlet>) ■ ■■■ core/ ■ ■ ■■■ auth/ ■ ■ ■ ■■■ auth.service.ts # Session signals, login, logout, quick role switcher ■ ■ ■ ■■■ auth.guard.ts # Route guard requiring valid session ■ ■ ■ ■■■ role.guard.ts # Role-based route guard (SUPERVISOR only) ■ ■ ■ ■■■ user.model.ts # User & AuthSession interfaces ■ ■ ■■■ repositories/ ■ ■ ■ ■■■ inspection.repository.ts # Repository interface ■ ■ ■ ■■■ local-inspection.repository.ts # Offline cache & persistence ■ ■ ■ ■■■ remote-inspection.repository.ts# Mock backend REST API ■ ■ ■ ■■■ sync-inspection.repository.ts # Unified offline-first repository ■ ■ ■ ■■■ inspection.adapter.ts # DTO <-> Entity <-> Domain adapter ■ ■ ■■■ services/ ■ ■ ■ ■■■ camera.service.ts # Capacitor Camera + web fallback handler ■ ■ ■ ■■■ network.service.ts# Capacitor Network + offline simulation ■ ■ ■ ■■■ toast.service.ts # Ionic Toast alerts ■ ■ ■ ■■■ mock-data.ts # Seed assets, users, and inspections ■ ■ ■■■ sync/ ■ ■ ■■■ sync-queue.service.ts # FIFO queue, retry policy, auto-flush ■ ■ ■■■ sync.model.ts # Sync queue item & status models ■ ■■■ shared/ ■ ■ ■■■ models/inspection.model.ts # Domain models, statuses, audit interfaces ■ ■ ■■■ pipes/status-color.pipe.ts # Maps statuses to Ionic theme colors ■ ■ ■■■ components/ ■ ■ ■■■ sync-badge/ # Connectivity badge & sync action sheet ■ ■ ■■■ image-preview-modal/# Full-screen image preview with metadata ■ ■ ■■■ qr-scanner-modal/ # Animated QR scanner & asset picker ■ ■ ■■■ signature-pad/ # HTML5 canvas touch/mouse signature pad ■ ■■■ features/ ■ ■■■ auth/login/ # Login page with demo one-tap accounts ■ ■■■ dashboard/ # Dashboard with KPI cards & audit timeline ■ ■■■ inspections/ ■ ■ ■■■ inspection-list/ # Search, filter segments, pull-refresh, pagination ■ ■ ■■■ inspection-form/ # Reactive form, conditional validation, camera ■ ■ ■■■ inspection-details/# Full inspection summary, photos, audit history ■ ■■■ approvals/approval-list# Dedicated Supervisor Review portal
```

10. Testing & Quality Assurance

The project includes a comprehensive Vitest test suite executing 15 automated unit tests:

• **inspection.adapter.spec.ts**: Tests full-fidelity transformation between remote API DTOs, domain models, and local storage entities.
• **auth.service.spec.ts**: Tests user authentication, role switching, session persistence, and logout cleanup.
• **sync-inspection.repository.spec.ts**: Tests offline draft saving, offline submission queueing, and supervisor approval/rejection logging.
• **status-color.pipe.spec.ts**: Tests status color mapping across all workflow, sync, and condition states.
• **app.component.spec.ts**: Tests root application initialization.

TEST EXECUTION COMMAND: Execute npm test in terminal to run all 15 tests. They complete in ~5 seconds with 100% pass rate.

11. Developer Quick-Start Reference

Common CLI Commands:

```
# Start development server at http://localhost:4200/ npm start # Run unit test suite npm test # Compile optimized production build to www/ npm run build # Add native mobile targets (requires Android Studio / Xcode) npx cap add android npx cap add ios npx cap sync
```